ITBDA4-14 Big Data Analytics Techniques

Project 2

Name	Sherman Opperman
Student number	CON-1375611-S5S8
Lecturer	Gilbert Lekula
Campus	Tyger Valley
GitHub repository	https://github.com/Opzalas369/ITBDA4-14_Big_Data_Analytics_Project_2

Question 1: MapReduce keyword location search

1.1 Logical layout

The diagram shows the six stages the job goes through for the keyword "africa". Only the documents that contain the keyword are drawn in the middle stages. The numbers are the real results of the run shown in section 1.4.

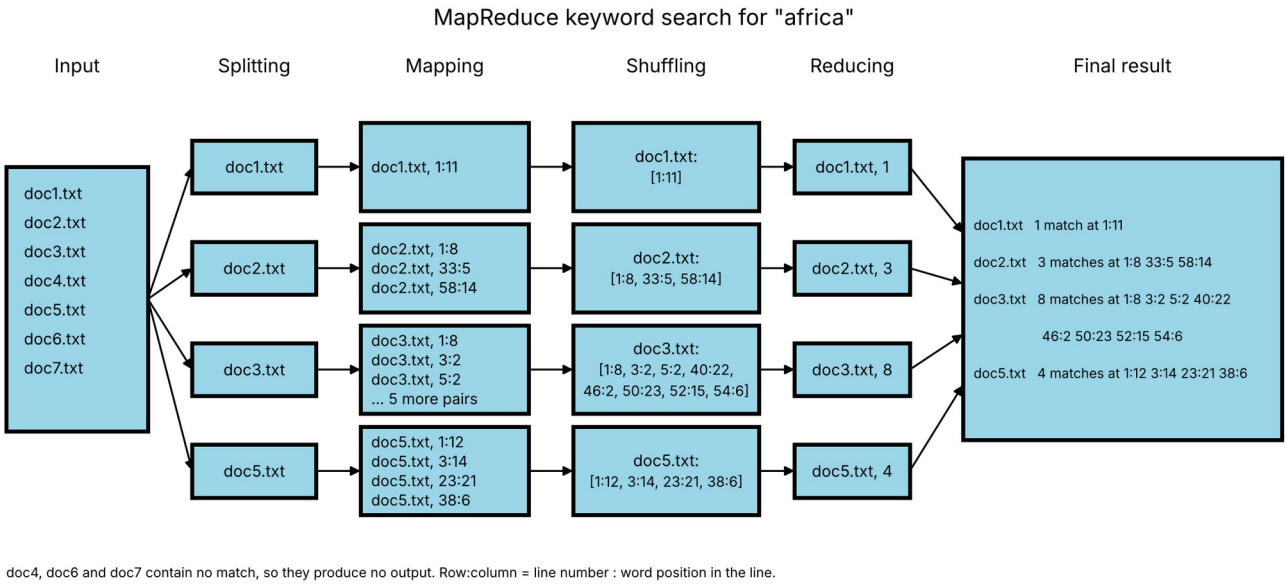


Figure 1: Logical layout of the MapReduce job for the keyword "africa"

- Input: the seven documents (doc1.txt to doc7.txt) are stored in HDFS.
- Splitting: each document becomes one split, so each document is handled by its own Mapper.
- Mapping: the Mapper reads its document line by line and writes (document, row:column) for every word that equals the keyword.
- Shuffling: Hadoop groups all the pairs with the same document name together and sends them to the Reducer.
- Reducing: the Reducer sorts the locations of each document by row and then column, and counts them.
- Final result: one line per document with the number of matches and their row:column positions.

1.2 Mapper

The Mapper is run once per document. It reads the keyword from the job configuration, so nothing is hard-coded. For every line it counts the row, cleans the line, splits it into words and writes one pair for every word that equals the keyword.

```
import java.io.IOException;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.lib.input.FileSplit;

// Q1.2 Mapper: finds the keyword in every line of a document and emits <document, row:column>
public class Q1_2_Mapper extends Mapper<Object, Text, Text, Text> {

    private String keyword;
    private Text fileName = new Text();
    private int row = 0;

    @Override
    protected void setup(Context context) {
        // the keyword comes from the job configuration, so it is not hard-coded
        keyword = context.getConfiguration().get("keyword").toLowerCase();
        // name of the document this Mapper is reading
        fileName.set(((FileSplit) context.getInputSplit()).getPath().getName());
    }

    @Override
    public void map(Object key, Text value, Context context)
            throws IOException, InterruptedException {
        row++;                                                  // current line (row)
        String line = value.toString().toLowerCase().replaceAll("(?U)^\\W+", "");
        String[] words = line.split("(?U)\\W+");                // words in the line

        for (int col = 0; col < words.length; col++) {
            if (words[col].equals(keyword)) {
                context.write(fileName, new Text(row + ":" + (col + 1)));   // <document,
row:column>
            }
        }
    }
}
```

- setup() runs once: it reads the keyword, changes it to lower case, and gets the name of the document from the input split.
- map() runs for every line: row++ gives the row number, and the line is changed to lower case so the search is not case sensitive.
- The line is split on anything that is not a letter or a digit, and the index in the word array plus one is the column.
- The output pair is (document name, "row:column"), for example (doc2.txt, 33:5).

1.3 Reducer

After the shuffle, the Reducer receives one document name together with all of its row:column locations. It sorts them and writes a single line for that document.

```
import java.io.IOException;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;

// Q1.3 Reducer: receives one document with all its row:column locations and writes one line
public class Q1_3_Reducer extends Reducer<Text, Text, Text, Text> {

    @Override
    public void reduce(Text key, Iterable<Text> values, Context context)
            throws IOException, InterruptedException {
        List<String> locations = new ArrayList<String>();
        for (Text value : values) {                    // every row:column found in this
document
            locations.add(value.toString());
        }

        // Hadoop does not keep the values in the order they were found, so sort by row then
column
        Collections.sort(locations, (a, b) -> {
            String[] x = a.split(":");
            String[] y = b.split(":");
            int byRow = Integer.compare(Integer.parseInt(x[0]), Integer.parseInt(y[0]));
            return byRow != 0 ? byRow : Integer.compare(Integer.parseInt(x[1]),
Integer.parseInt(y[1]));
        });

        context.write(key, new Text(locations.size() + " match(es) at " + String.join(" ",
locations)));
    }
}
```

- Hadoop does not keep the values in the order they were found, so the Reducer sorts them by row first and then by column.
- The output line holds the document name, the number of matches, and the locations, for example "doc2.txt 3 match(es) at 1:8 33:5 58:14".

1.4 Hadoop configuration

The Driver class sets up the job. It receives the input folder, the output folder and the keyword as arguments, which is what makes the job dynamic.

```
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

// Q1.4 Driver: Hadoop MapReduce configuration settings
public class Q1_4_Driver {

    public static void main(String[] args) throws Exception {
        // args[0] = input folder, args[1] = output folder, args[2] = keyword
        if (args.length != 3) {
            System.err.println("Usage: Q1_4_Driver <input folder> <output folder> <keyword>");
            System.exit(2);
        }
        Configuration conf = new Configuration();
        conf.set("keyword", args[2]);                          // keyword passed in
dynamically

        Job job = Job.getInstance(conf, "keyword location search");
        job.setJarByClass(Q1_4_Driver.class);
        job.setMapperClass(Q1_2_Mapper.class);
        job.setReducerClass(Q1_3_Reducer.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(Text.class);

        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileInputFormat.setMinInputSplitSize(job, Long.MAX_VALUE);   // one Mapper per
document, so row numbers stay correct
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

- conf.set("keyword", args[2]) passes the keyword to every Mapper.
- setMinInputSplitSize with Long.MAX_VALUE stops Hadoop from cutting a document into pieces, so one Mapper reads one whole document and its row numbers are correct.
- The Mapper and Reducer classes are registered, and both the output key and the output value are Text.
- Environment: Hadoop 3.2.4 on Windows with Java 8, running HDFS and YARN on one machine (pseudo-distributed mode). Hadoop was started from an Administrator window, because the ResourceManager failed with "Access is denied" in a normal window.
- YARN marked the node unhealthy because the C: drive was over 90% full, so the limit yarn.nodemanager.disk-health-checker.max-disk-utilization-per-disk-percentage was raised to 99 in yarn-site.xml.

1.4 Demonstration on Hadoop

Step 1: Hadoop is started and the four services are checked with jps. NameNode and DataNode belong to HDFS, ResourceManager and NodeManager belong to YARN.

```
starting yarn daemons
PS C:\Users\sherm\Downloads\project2_step1_Q1 (2)\project2\Q1> Start-Sleep -Seconds 15
PS C:\Users\sherm\Downloads\project2_step1_Q1 (2)\project2\Q1> jps
37104 NodeManager
3492 Jps
40180 ResourceManager
53188 DataNode
52956 NameNode
```

Figure 2: jps shows NameNode, DataNode, ResourceManager and NodeManager running

```
PS C:\Users\sherm\Downloads\project2_step1_Q1 (2)> hadoop version
Hadoop 3.2.4
Source code repository Unknown -r 7e5d9983b388e372fe640f21f048f2f2ae6e9eba
Compiled by ubuntu on 2022-07-12T11:58Z
Compiled with protoc 2.5.0
From source with checksum ee031c16fe785bbb35252c749418712
This command was run using /C:/hadoopsetup/hadoop-3.2.4/share/hadoop/common/hadoop-common-3.2.4.jar
PS C:\Users\sherm\Downloads\project2_step1_Q1 (2)>
```

Figure 3: The installed Hadoop version

Step 2: The three classes are compiled against the Hadoop libraries and packed into q1.jar.

```
$cp = hadoop classpath
javac -cp $cp -d build Q1_2_Mapper.java Q1_3_Reducer.java Q1_4_Driver.java
jar cf q1.jar -C build .
```

```
PS C:\Users\sherm\Downloads\project2_step1_Q1 (2)\project2\Q1> Remove-Item -Recurse -Force build, q1.jar -ErrorAction SilentlyContinue
PS C:\Users\sherm\Downloads\project2_step1_Q1 (2)\project2\Q1> mkdir build


    Directory: C:\Users\sherm\Downloads\project2_step1_Q1 (2)\project2\Q1


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        2026/10/29     20:38                build


PS C:\Users\sherm\Downloads\project2_step1_Q1 (2)\project2\Q1> $cp = hadoop classpath
PS C:\Users\sherm\Downloads\project2_step1_Q1 (2)\project2\Q1> javac -cp $cp -d build Q1_2_Mapper.java Q1_3_Reducer.java Q1_4_Driver.java
PS C:\Users\sherm\Downloads\project2_step1_Q1 (2)\project2\Q1> jar cf q1.jar -C build .
PS C:\Users\sherm\Downloads\project2_step1_Q1 (2)\project2\Q1> dir q1.jar


    Directory: C:\Users\sherm\Downloads\project2_step1_Q1 (2)\project2\Q1


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----        2026/10/29     20:39           4053 q1.jar
```

Figure 4: Compiling and building q1.jar

Step 3: The seven documents are copied into HDFS. This copy is the input of the job.

```
hdfs dfs -mkdir -p /user/sherm/docs
Get-ChildItem ..\datasets\map_reduce_test_docs\*.txt | ForEach-Object { hdfs dfs -put -f
$_.FullName /user/sherm/docs }
hdfs dfs -ls /user/sherm/docs
```

```
19272 DataNode
PS C:\Users\sherm\Downloads\project2_step1_Q1 (2)\project2\Q1> hdfs dfs -ls /user/sherm/docs
Found 7 items
-rw-r--r--   1 sherm supergroup       4202 2026-10-29 20:40 /user/sherm/docs/doc1.txt
-rw-r--r--   1 sherm supergroup       4301 2026-10-29 20:40 /user/sherm/docs/doc2.txt
-rw-r--r--   1 sherm supergroup       4490 2026-10-29 20:40 /user/sherm/docs/doc3.txt
-rw-r--r--   1 sherm supergroup       1649 2026-10-29 20:40 /user/sherm/docs/doc4.txt
-rw-r--r--   1 sherm supergroup       4020 2026-10-29 20:40 /user/sherm/docs/doc5.txt
-rw-r--r--   1 sherm supergroup       2125 2026-10-29 20:40 /user/sherm/docs/doc6.txt
-rw-r--r--   1 sherm supergroup       2643 2026-10-29 20:40 /user/sherm/docs/doc7.txt
PS C:\Users\sherm\Downloads\project2_step1_Q1 (2)\project2\Q1>
```

Figure 5: The seven input documents in HDFS

Step 4: The job is run with the keyword africa. The three arguments are the input folder, the output folder and the keyword.

```
hadoop jar q1.jar Q1_4_Driver /user/sherm/docs /user/sherm/out_africa africa
```

```
PS C:\Users\sherm\Downloads\project2_step1_Q1 (2)\project2\Q1> hdfs dfs -rm -r -f /user/sherm/out_africa
PS C:\Users\sherm\Downloads\project2_step1_Q1 (2)\project2\Q1> hadoop jar q1.jar Q1_4_Driver /user/sherm/docs /user/sherm/out_africa africa
2026-10-29 21:10:08,630 INFO client.RMProxy: Connecting to ResourceManager at /0.0.0.0:8032
2026-10-29 21:10:09,506 WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not performed. Implement the Tool interface and execute your app
2026-10-29 21:10:09,536 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn/staging/sherm/.staging/job_1793300921433_0001
2026-10-29 21:10:10,370 INFO input.FileInputFormat: Total input files to process : 7
2026-10-29 21:10:10,520 INFO mapreduce.JobSubmitter: number of splits:7
2026-10-29 21:10:10,694 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1793300921433_0001
2026-10-29 21:10:10,698 INFO mapreduce.JobSubmitter: Executing with tokens: []
2026-10-29 21:10:10,936 INFO conf.Configuration: resource-types.xml not found
2026-10-29 21:10:10,937 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2026-10-29 21:10:11,454 INFO impl.YarnClientImpl: Submitted application application_1793300921433_0001
2026-10-29 21:10:11,523 INFO mapreduce.Job: The url to track the job: http://Opzalas:8088/proxy/application_1793300921433_0001/
2026-10-29 21:10:11,526 INFO mapreduce.Job: Running job: job_1793300921433_0001
2026-10-29 21:10:27,067 INFO mapreduce.Job: Job job_1793300921433_0001 running in uber mode : false
2026-10-29 21:10:27,070 INFO mapreduce.Job:  map 0% reduce 0%
2026-10-29 21:10:56,062 INFO mapreduce.Job:  map 86% reduce 0%
2026-10-29 21:11:09,508 INFO mapreduce.Job:  map 100% reduce 0%
2026-10-29 21:11:10,549 INFO mapreduce.Job:  map 100% reduce 100%
2026-10-29 21:11:11,624 INFO mapreduce.Job: Job job_1793300921433_0001 completed successfully
2026-10-29 21:11:12,512 INFO mapreduce.Job: Counters: 54
```

Figure 6: The job for "africa": 7 splits, map 100%, reduce 100%, completed successfully

```
Map-Reduce Framework
        Map input records=307
        Map output records=16
        Map output bytes=225
        Map output materialized bytes=299
        Input split bytes=777
        Combine input records=0
        Combine output records=0
        Reduce input groups=4
        Reduce shuffle bytes=299
        Reduce input records=16
        Reduce output records=4
```

Figure 7: Counters for "africa": 307 lines read, 16 matches found, 4 documents in the output

Step 5: The result is read from the output folder.

```
hdfs dfs -cat /user/sherm/out_africa/part-r-00000
```

```
                Bytes Written=177
PS C:\Users\sherm\Downloads\project2_step1_Q1 (2)\project2\Q1> hdfs dfs -cat /user/sherm/out_africa/part-r-00000
doc1.txt        1 match(es) at 1:11
doc2.txt        3 match(es) at 1:8 33:5 58:14
doc3.txt        8 match(es) at 1:8 3:2 5:2 40:22 46:2 50:23 52:15 54:6
doc5.txt        4 match(es) at 1:12 3:14 23:21 38:6
PS C:\Users\sherm\Downloads\project2_step1_Q1 (2)\project2\Q1> hdfs dfs -cat /user/sherm/out_africa/part-r-00000 | Out-File -Encoding utf8 Q1_output_africa.txt
PS C:\Users\sherm\Downloads\project2_step1_Q1 (2)\project2\Q1>
```

Figure 8: Result for "africa"

Document	Matches	Row:column
doc1.txt	1	1:11
doc2.txt	3	1:8 33:5 58:14
doc3.txt	8	1:8 3:2 5:2 40:22 46:2 50:23 52:15 54:6
doc5.txt	4	1:12 3:14 23:21 38:6

The counters agree with the result: 16 pairs came out of the Mappers (1 + 3 + 8 + 4), and the Reducer grouped them into 4 documents. doc4, doc6 and doc7 do not contain the word, so they do not appear.

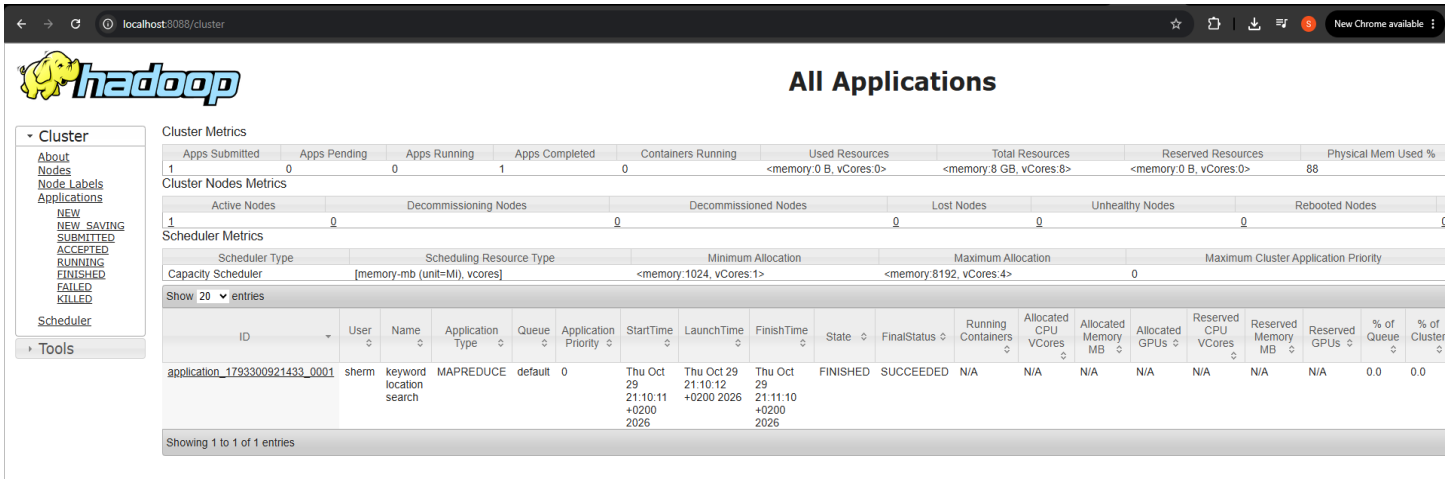


Figure 9: The YARN web page shows the application as FINISHED and SUCCEEDED

Step 6: To show that the job is dynamic, the same jar is run again with another keyword and a new output folder. Nothing in the code was changed.

```
hadoop jar q1.jar Q1_4_Driver /user/sherm/docs /user/sherm/out_SARS SARS
```

```
PS C:\Users\sherm\Downloads\project2_step1_Q1 (2)\project2\Q1> hdfs dfs -rm -r -f /user/sherm/out_SARS
PS C:\Users\sherm\Downloads\project2_step1_Q1 (2)\project2\Q1> hadoop jar q1.jar Q1_4_Driver /user/sherm/docs /user/sherm/out_SARS SARS
2026-10-29 21:14:25,099 INFO client.RMProxy: Connecting to ResourceManager at /0.0.0.0:8032
2026-10-29 21:14:25,907 WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not performed. Implement the Tool interface and execute your app
2026-10-29 21:14:25,927 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn/staging/sherm/.staging/job_1793300921433_0002
2026-10-29 21:14:26,149 INFO input.FileInputFormat: Total input files to process : 7
2026-10-29 21:14:26,261 INFO mapreduce.JobSubmitter: number of splits:7
2026-10-29 21:14:26,380 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1793300921433_0002
2026-10-29 21:14:26,384 INFO mapreduce.JobSubmitter: Executing with tokens: []
2026-10-29 21:14:26,568 INFO conf.Configuration: resource-types.xml not found
2026-10-29 21:14:26,568 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2026-10-29 21:14:26,661 INFO impl.YarnClientImpl: Submitted application application_1793300921433_0002
2026-10-29 21:14:26,702 INFO mapreduce.Job: The url to track the job: http://Opzalas:8088/proxy/application_1793300921433_0002/
2026-10-29 21:14:26,703 INFO mapreduce.Job: Running job: job_1793300921433_0002
2026-10-29 21:14:34,851 INFO mapreduce.Job: Job job_1793300921433_0002 running in uber mode : false
2026-10-29 21:14:34,852 INFO mapreduce.Job:  map 0% reduce 0%
2026-10-29 21:14:47,111 INFO mapreduce.Job:  map 86% reduce 0%
2026-10-29 21:14:51,158 INFO mapreduce.Job:  map 100% reduce 0%
2026-10-29 21:14:53,182 INFO mapreduce.Job:  map 100% reduce 100%
2026-10-29 21:14:54,200 INFO mapreduce.Job: Job job_1793300921433_0002 completed successfully
2026-10-29 21:14:54,294 INFO mapreduce.Job: Counters: 54
```

Figure 10: The same job for the keyword "SARS"

```
Map input records=307
Map output records=11
Map output bytes=156
Map output materialized bytes=220
Input split bytes=777
Combine input records=0
Combine output records=0
Reduce input groups=1
Reduce shuffle bytes=220
Reduce input records=11
Reduce output records=1
```

Figure 11: Counters for "SARS": 11 matches, 1 document

```
hdfs dfs -cat /user/sherm/out_SARS/part-r-00000
```

```
                Bytes Written=82
PS C:\Users\sherm\Downloads\project2_step1_Q1 (2)\project2\Q1> hdfs dfs -cat /user/sherm/out_SARS/part-r-00000
doc1.txt        11 match(es) at 3:6 5:9 9:11 11:8 17:29 19:2 33:2 42:1 46:25 52:16 53:22
PS C:\Users\sherm\Downloads\project2_step1_Q1 (2)\project2\Q1> hdfs dfs -cat /user/sherm/out_SARS/part-r-00000 | Out-File -Encoding utf8 Q1_output_SARS.txt
PS C:\Users\sherm\Downloads\project2_step1_Q1 (2)\project2\Q1>
```

Figure 12: Result for "SARS": all 11 matches are in doc1.txt

The same Mapper and Reducer were used for both keywords. Only the arguments on the command line changed, and because the keyword and every line are converted to lower case before they are compared, the search does not depend on how the word is capitalised.

Question 2: Topic modelling of the news articles

2.1 Data frame and denoising

Each article is read into one row of a data frame. The first column (file_name) holds the file name and the second column (content) holds the full text. A third column (denoised) holds the cleaned text that is used for the model.

```
library(tm)
library(topicmodels)
library(ggplot2)
library(wordcloud)

data_dir <- "../datasets/sa_news"
files <- list.files(data_dir, pattern = "\\.txt$", full.names = TRUE)
if (length(files) == 0) stop("No .txt files found. setwd() to the Q2 folder first.")
files <- files[order(as.numeric(gsub("\\D", "", basename(files))))]

news_df <- data.frame(
  file_name = basename(files),
  content   = sapply(files, function(f) paste(readLines(f, warn = FALSE, encoding = "UTF-8"),
collapse = " ")),
  stringsAsFactors = FALSE
)
rownames(news_df) <- NULL

# words that carry no topic information in these articles
noise <- c(stopwords("en"), "said", "also", "will", "may", "sanews", "south", "africa",
           "african", "including", "expected", "gov",
           month.name, tolower(month.name), "monday", "tuesday", "wednesday", "thursday",
"friday",
           "saturday", "sunday")

denoise <- function(text) {
  text <- tolower(text)
  text <- gsub("[^a-z ]", " ", text)            # keep letters only (removes numbers and
punctuation)
  words <- strsplit(text, "\\s+")[[1]]
  words <- words[nchar(words) > 2 & !(words %in% noise)]
  paste(words, collapse = " ")
}
news_df$denoised <- sapply(news_df$content, denoise)

# show the start of each article before and after denoising
print(data.frame(file_name = news_df$file_name,
                 content   = substr(news_df$content, 1, 60),
                 denoised  = substr(news_df$denoised, 1, 60)))
write.csv(news_df, "Q2_1_dataframe.csv", row.names = FALSE)
```

The denoising function does the following:

- changes all text to lower case, so "Eskom" and "eskom" are the same word;
- removes numbers and punctuation, keeping letters only;
- removes English stop words (such as "the" and "and"), month and day names, and words that appear in almost every article and say nothing about a topic ("said", "south", "africa", "sanews" and similar);
- removes words with two letters or fewer.

The ten articles contained 3,832 words before cleaning and 2,101 words after, so about 45% of the text was noise. The table shows the start of each article before and after cleaning.

file_name	content (start)	denoised (start)
news_1.txt	Eskom former official, service provider appear i…	eskom former official service provider appear co…
news_2.txt	Gauteng Education commits to implementing Mayibu…	gauteng education commits implementing mayibuye…
news_3.txt	New Labour Minister charts the way forward Tues…	new labour minister charts way forward newly app…
news_4.txt	Police recue two kidnapped victims in Benoni Tue…	police recue two kidnapped victims benoni swift…
news_5.txt	President Ramaphosa sends condolences to decease…	president ramaphosa sends condolences deceased s…
news_6.txt	Accounting officers must be accountable for non-…	accounting officers must accountable non payment…
news_7.txt	Democratic dispensation necessitated a complete…	democratic dispensation necessitated complete ov…
news_8.txt	Trust and relationships key ingredients to coali…	trust relationships key ingredients coalition go…
news_9.txt	PSC applauds officials who refuse to accept a br…	psc applauds officials refuse accept bribes publ…
news_10.txt	Wet, cold weather expected to continue in Wester…	wet cold weather continue western cape cold wet…

2.2 LDA topic model

LDA treats every article as a mixture of topics and every topic as a mixture of words. The cleaned text is turned into a document-term matrix, words that appear only once are dropped, and a model with three topics (k = 3) is fitted with Gibbs sampling. Three topics suit ten short articles, because more topics would leave only a few documents in each. A fixed seed makes the result repeatable. The top 10 words per topic (top-k = 10) come from the word distribution of the model, and the topic mix of each article comes from the document distribution.

```
corpus <- VCorpus(VectorSource(news_df$denoised))
dtm <- DocumentTermMatrix(corpus)
dtm <- dtm[, colSums(as.matrix(dtm)) >= 2]       # drop words that appear only once

k <- 3          # number of topics
top_k <- 10     # words shown per topic
lda <- LDA(dtm, k = k, method = "Gibbs", control = list(seed = 1234, burnin = 500, iter =
2000))

word_dist <- posterior(lda)$terms                # topic x word probabilities
doc_dist  <- posterior(lda)$topics               # document x topic probabilities

top_terms <- do.call(rbind, lapply(1:k, function(t) {
  p <- sort(word_dist[t, ], decreasing = TRUE)[1:top_k]
  data.frame(topic = paste("Topic", t), word = names(p), probability = round(as.numeric(p),
4))
}))
print(top_terms)

doc_topics <- data.frame(file_name = news_df$file_name, round(doc_dist, 3))
colnames(doc_topics)[-1] <- paste("Topic", 1:k)
doc_topics$main_topic <- paste("Topic", apply(doc_dist, 1, which.max))
print(doc_topics)

write.csv(top_terms, "Q2_2_top_words.csv", row.names = FALSE)
write.csv(doc_topics, "Q2_2_document_topics.csv", row.names = FALSE)
```

Top 10 words of each topic, with the probability of the word in that topic (the word distribution):

Rank	Topic 1	Topic 2	Topic 3
1	eskom (0.027)	invoices (0.0313)	government (0.0356)
2	police (0.0248)	public (0.0294)	department (0.025)
3	department (0.0225)	service (0.0294)	coalition (0.0232)
4	cape (0.0181)	days (0.0255)	national (0.0232)
5	cold (0.0158)	psc (0.0235)	administration (0.0215)
6	weather (0.0158)	gxoyiya (0.0216)	country (0.0179)
7	west (0.0158)	departments (0.0196)	public (0.0179)
8	western (0.0158)	report (0.0196)	governments (0.0161)
9	sandf (0.0136)	management (0.0177)	power (0.0144)
10	service (0.0136)	officials (0.0138)	role (0.0144)

Topic mix of each article (the document distribution). Each row adds up to 1, and the main topic is the largest value:

Article	Topic 1	Topic 2	Topic 3	Main topic
news_1.txt	0.494	0.242	0.265	Topic 1
news_2.txt	0.203	0.613	0.184	Topic 2
news_3.txt	0.175	0.131	0.695	Topic 3
news_4.txt	0.633	0.191	0.176	Topic 1
news_5.txt	0.694	0.153	0.153	Topic 1
news_6.txt	0.134	0.761	0.106	Topic 2
news_7.txt	0.166	0.522	0.312	Topic 2
news_8.txt	0.08	0.105	0.815	Topic 3
news_9.txt	0.143	0.632	0.225	Topic 2
news_10.txt	0.707	0.111	0.182	Topic 1

What the topics mean:

- Topic 1: Eskom, police, weather and the SANDF. This is crime, emergency and weather news. Its main articles are the Eskom court case (news_1), the Benoni kidnapping rescue (news_4), the SANDF deaths (news_5) and the Western Cape cold weather (news_10).
- Topic 2: invoices, public service, PSC and Gxoyiya. This is public service accountability. Its main articles are the Mayibuye school report (news_2), non-payment of invoices (news_6), public administration reform (news_7) and bribery at ports of entry (news_9).
- Topic 3: government, coalition, national and administration. This is government, labour and coalition governance. Its main articles are the new Labour Minister (news_3) and coalition governments (news_8).

2.3 Visualisations

Two visualisations are used. The code is below.

```
# Visualisation 1: top words of each topic (bar chart)
top_terms$label <- reorder(paste(top_terms$topic, top_terms$word, sep = "__"),
top_terms$probability)
bar_chart <- ggplot(top_terms, aes(x = label, y = probability, fill = topic)) +
  geom_col(show.legend = FALSE) +
  scale_x_discrete(labels = function(x) sub("^.*__", "", x)) +
  coord_flip() +
  facet_wrap(~ topic, scales = "free_y") +
  labs(title = "Top 10 words in each topic", x = "Word", y = "Probability in topic")
ggsave("Q2_3_bar_chart.png", bar_chart, width = 10, height = 5, dpi = 150)
print(bar_chart)

# Visualisation 2: word clouds, one per topic
png("Q2_3_wordclouds.png", width = 1500, height = 500, res = 130)
par(mfrow = c(1, k), mar = c(0, 0, 2, 0))
for (t in 1:k) {
  p <- sort(word_dist[t, ], decreasing = TRUE)[1:30]
  wordcloud(names(p), p, scale = c(2.2, 0.5), random.order = FALSE, colors = c("#1B9E77",
"#D95F02", "#7570B3")[t])
  title(paste("Topic", t))
}
dev.off()
```

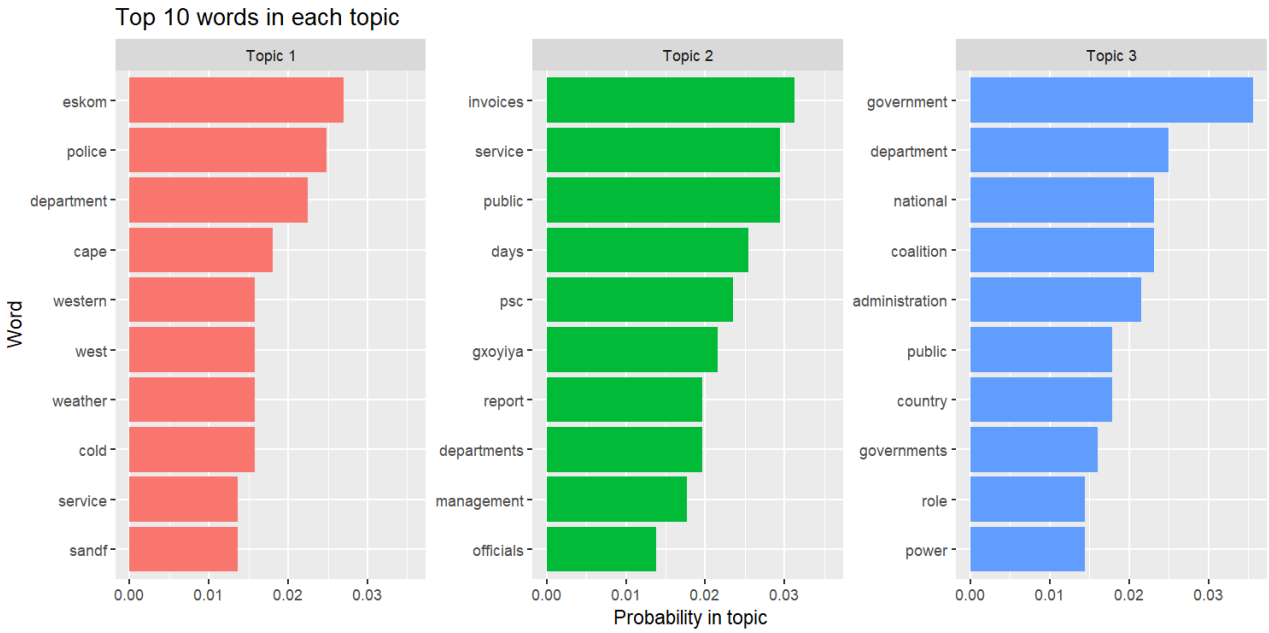


Figure 13: Bar chart of the top 10 words in each topic

The bar chart shows the words with the highest probability in each topic. The length of a bar is the probability of the word, so it is easy to compare the words inside one topic. It confirms the topic meanings: "eskom" and "police" lead Topic 1, "invoices" and "service" lead Topic 2, and "government" leads Topic 3. The words "public" and "department" appear in more than one topic, which shows that the topics overlap a little.

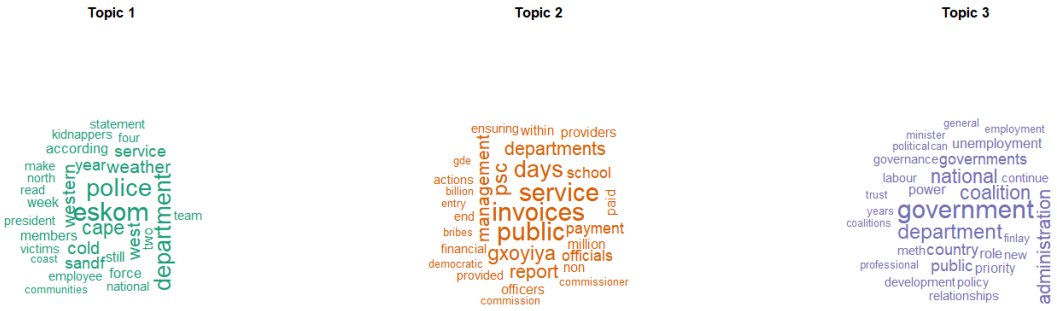


Figure 14: Word clouds of the top 30 words in each topic

The word clouds show the top 30 words of each topic, and a bigger word means a higher probability. They give a quick picture of what each topic is about and show more words than the bar chart, such as "kidnappers" and "weather" in Topic 1, "bribes" and "payment" in Topic 2, and "coalition" and "unemployment" in Topic 3.

Question 3: Web scraping and text summarisation

3.1 Web scraping and fair use

The script reads the qualifications page, collects the link to every qualification, and then opens each qualification page and keeps its title and its paragraphs of text. The result is saved in Q3_1_qualifications.csv.

```
library(rvest)
library(httr)
library(stringr)

base_url <- "https://www.eduvos.com"
ua <- user_agent("EduvosStudentProject/1.0 (non-commercial, educational)")

# fair use: read robots.txt, identify ourselves, and wait between requests
cat(httr::content(GET(paste0(base_url, "/robots.txt"), ua), "text", encoding = "UTF-8"))

get_page <- function(url) {
  Sys.sleep(2)
  read_html(GET(url, ua))
}

# the qualifications page links to one page per qualification
list_page <- get_page(paste0(base_url, "/qualifications/"))
links <- html_attr(html_elements(list_page, "a[href*='/programmes/']"), "href")
links <- unique(links[grepl("/programmes/.+", links) & !grepl("\\?", links)])
links <- ifelse(startsWith(links, "http"), links, paste0(base_url, links))

# open each qualification page and keep its title and paragraphs
qualifications <- data.frame(url = links, title = NA, text = NA)
for (i in seq_along(links)) {
  page <- tryCatch(get_page(links[i]), error = function(e) NULL)
  if (is.null(page)) { cat("Skipped (page not found):", links[i], "\n"); next }
  paragraphs <- html_text2(html_elements(page, "p"))
  qualifications$title[i] <- html_text2(html_element(page, "h1"))
  qualifications$text[i] <- paste(paragraphs[nchar(paragraphs) > 60], collapse = " ")
}
qualifications <- qualifications[!is.na(qualifications$text), ]
write.csv(qualifications, "Q3_1_qualifications.csv", row.names = FALSE)
```

The robots.txt file of the website, as printed by the script:

```
User-agent: *
Allow: /

User-agent: GPTBot
Allow: /

(the same "Allow: /" rule is repeated for ChatGPT-User, OAI-SearchBot,
Google-Extended, ClaudeBot and PerplexityBot)

Sitemap: https://www.eduvos.com/sitemap-ai.xml
Sitemap: https://www.eduvos.com/sitemap.xml
```

The line printed for the one page that was not found:

```
Skipped (page not found): https://www.eduvos.com/programmes/bachelor-of-science-honours-in-
information-
technology-security-and-network-engineering/
```

How the scraping respects fair use:

- The script first reads the robots.txt file of the website and prints it. The file of Eduvos allows all crawlers to read the site and has no Disallow rule, so the qualification pages may be read.

- Every request carries a user agent that names the project and says it is non-commercial and educational, so the website owner can see who is reading the pages.
- The script waits 2 seconds before every request (Sys.sleep), so the server is never put under load.
- Only public pages are read, each page is read once, and only the text is kept. Nothing behind a login is used.
- The data is used only for this assignment and is not published or sold. The text belongs to Eduvos, and the repository contains only the scraped file needed to mark the work.

Duplicate links that only differ in the delivery mode (the ones with "?id=" in the address) are skipped, and one programme page returned an HTTP 404 error. The script catches the error, prints "Skipped (page not found)" and carries on. The scrape therefore gave 66 qualification pages, which is the 66 rows of the data frame.

3.2 Text preprocessing

The text of all 66 pages is joined and split into sentences, and each sentence is cleaned. The steps are:

- Bullet points are changed to full stops, so every bullet becomes its own sentence, and duplicate sentences are removed.
- Sentences with the word "credits" are dropped, because they are module lists and not descriptions.
- Stray endings left by the bullet points (such as ". ." or ";.") are tidied to a single full stop.
- Only sentences of 8 to 60 words are kept, which removes headings, short fragments and very long run-on lists.
- Each sentence is changed to lower case, all numbers and punctuation are removed, English stop words (such as "the" and "and") are removed, and words with two letters or fewer are removed.

```
library(tm)

# split the text into sentences (bullet points count as separate sentences)
text <- str_replace_all(paste(qualifications$text, collapse = " "), "[•■◦]", ". ")
sentences <- unique(unlist(str_split(text, "(?<=[.!?])\\s+(?=[A-Z])")))
sentences <- sentences[!grepl("credits", sentences)]               # drop the module lists
sentences <- str_replace_all(sentences, "[\\s.;]+\\.$", ".")        # remove stray endings
such as ". ." and ";."
sentences <- sentences[str_count(sentences, "\\S+") %in% 8:60]      # keep sentences of 8 to
60 words

# lower case, letters only, remove stop words and very short words
clean <- function(s) {
  w <- strsplit(gsub("[^a-z ]", " ", tolower(s)), "\\s+")[[1]]
  w[nchar(w) > 2 & !(w %in% stopwords("en"))]
}
tokens <- lapply(sentences, clean)

cat("Sentences:", length(sentences), "\n")
cat("Words before:", sum(str_count(sentences, "\\S+")), " after:", sum(lengths(tokens)), "\n")
cat("Before:", sentences[1], "\nAfter: ", paste(tokens[[1]], collapse = " "), "\n")
```

Result of the preprocessing:

Measure	Value
Qualification pages scraped	66
Sentences kept	485
Words before cleaning	10,653
Words after cleaning	6,917

Measure	Value
Different words left	1,470

Example of one sentence before and after cleaning:

	Text
Before	Eduvos is a private higher education institution in South Africa, registered with the DHET as Eduvos (Pty) Ltd (2001/HE07/008).
After	eduvos private higher education institution south africa registered dhet eduvos pty ltd

About 35% of the words were removed as noise, and the sentences are now ready to be scored.

3.3 Summarisation and visualisations

The summary of the whole set of qualification pages is built by extraction: the five best sentences are selected and kept in the order in which they appear. Two different techniques decide which sentences are the best.

- Technique 1, word frequency: every word gets a weight between 0 and 1 from how often it appears in all the text. A sentence is scored by the average weight of its words, so a sentence made of common topic words scores high.
- Technique 2, TextRank: every sentence is a node in a graph. Two sentences are linked when they use the same words (the link strength is the correlation of their word counts), and the PageRank algorithm then scores each sentence. A sentence scores high when it is similar to many other sentences that are also important.

```
library(igraph)

# Technique 1: word frequency. A sentence scores high when it uses many frequent words.
freq <- sort(table(unlist(tokens)), decreasing = TRUE)
weight <- as.numeric(freq) / max(freq)
names(weight) <- names(freq)
score_freq <- sapply(tokens, function(w) mean(weight[w]))

# Technique 2: TextRank. Sentences that are similar to many other sentences score high.
tf <- t(sapply(tokens, function(w) as.numeric(table(factor(w, levels = names(freq))))))
sim <- cor(t(tf))                                                  # similarity between
sentences
sim[is.na(sim) | sim < 0] <- 0
diag(sim) <- 0
score_rank <- page_rank(graph_from_adjacency_matrix(sim, mode = "undirected", weighted =
TRUE))$vector

# the 5 best sentences of each technique, in the order they appear
top5 <- function(score) sort(order(score, decreasing = TRUE)[1:5])
cat("\nSUMMARY 1: word frequency\n"); cat(paste("-", sentences[top5(score_freq)]), sep = "\n")
cat("\nSUMMARY 2: TextRank\n");       cat(paste("-", sentences[top5(score_rank)]), sep = "\n")
```

Summary 1 (word frequency):

- International students are welcome to apply for this programme.
- Students develop an advanced level of knowledge and skills in the field of accounting.
- Students develop an advanced level of knowledge and skills in the field of human resources.
- Students develop an intermediate level of knowledge and skills in marketing.
- Students develop an advanced level of knowledge and skills in the field of tourism.

Summary 2 (TextRank):

- It also provides an access pathway for students who do not meet the entry requirements for a degree and wish to apply for admission to an Eduvos Bachelor of Commerce programme after successfully completing the Higher Certificate, subject to the applicable admission requirements.
- Work Readiness Skills A and B support the development of critical thinking and practical workplace capabilities and enable students to apply knowledge acquired across the programme.
- Second-year-level modules are built on the foundation developed through the first-year modules, and students develop an intermediate level of knowledge in the field of business. * Students who pass the Human Resource Information Systems exams with 75% or more will receive a Sage payroll certificate.
- Students develop an advanced level of knowledge and skills in the field of accounting.
- Students develop an intermediate level of knowledge and skills in Human Resource Management. *Students who pass the Human Resource Information Systems exams with 75% or more will receive a Sage Payroll certificate.

Comparison of the two summaries:

- The word frequency summary picks short sentences that are full of the most common words ("students", "programme", "knowledge", "skills", "level"). It is easy to read, but it is repetitive: four of the five sentences are the same template with a different field of study, because a short sentence made of common words scores well.
- The TextRank summary picks longer sentences that say more. They describe the access pathway into a degree, the work readiness skills, and the structure of the years of study, which is a better picture of what the qualifications offer.
- Only one sentence is in both summaries (the accounting sentence). The scores of the two techniques are still strongly related (correlation of 0.93, see Figure 16), but the techniques disagree on the very top sentences.
- The same template sentences ("Students develop an advanced level of knowledge and skills in the field of ...") are written on many programme pages. Both techniques rate them highly because they repeat many common words, which shows how much of the text is shared between programmes.
- Two of the TextRank sentences carry a footnote that starts with "*" (about the Sage payroll certificate). The footnote is part of the text of those pages and is not split off, which is a small limitation of the simple sentence splitting.

The two visualisations are made with the code below.

```
library(ggplot2)

# Visualisation 1: the 15 most frequent words
top_words <- data.frame(word = names(freq)[1:15], count = as.numeric(freq)[1:15])
words_chart <- ggplot(top_words, aes(x = reorder(word, count), y = count)) +
  geom_col(fill = "#2C7FB8") + coord_flip() +
  labs(title = "15 most frequent words", x = "Word", y = "Count")
ggsave("Q3_3_top_words.png", words_chart, width = 8, height = 5)

# Visualisation 2: the sentence scores of the two techniques compared
scores_chart <- ggplot(data.frame(freq = score_freq, rank = score_rank), aes(x = freq, y =
rank)) +
  geom_point(colour = "#2C7FB8", alpha = 0.7) +
  labs(title = "Sentence scores of the two techniques", x = "Word frequency score", y =
"TextRank score")
ggsave("Q3_3_sentence_scores.png", scores_chart, width = 8, height = 5)
```

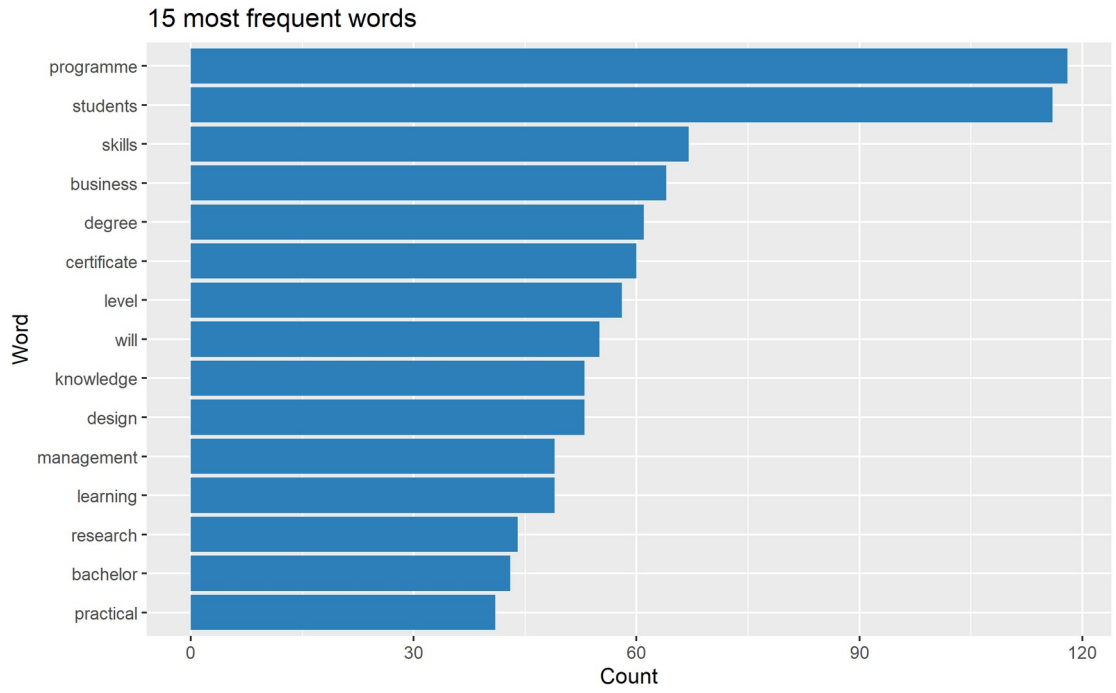


Figure 15: The 15 most frequent words in the cleaned text

The bar chart shows the 15 words with the highest counts, which are the words behind the word frequency scores. "programme" (118) and "students" (116) lead by a wide margin, followed by "skills", "business", "degree" and "certificate". The list shows that the text is about programmes, the students who take them, the skills they build, and the level of the qualification (degree, certificate, bachelor). The words "will" (55) and "level" (58) are frequent because of the repeated template sentences.

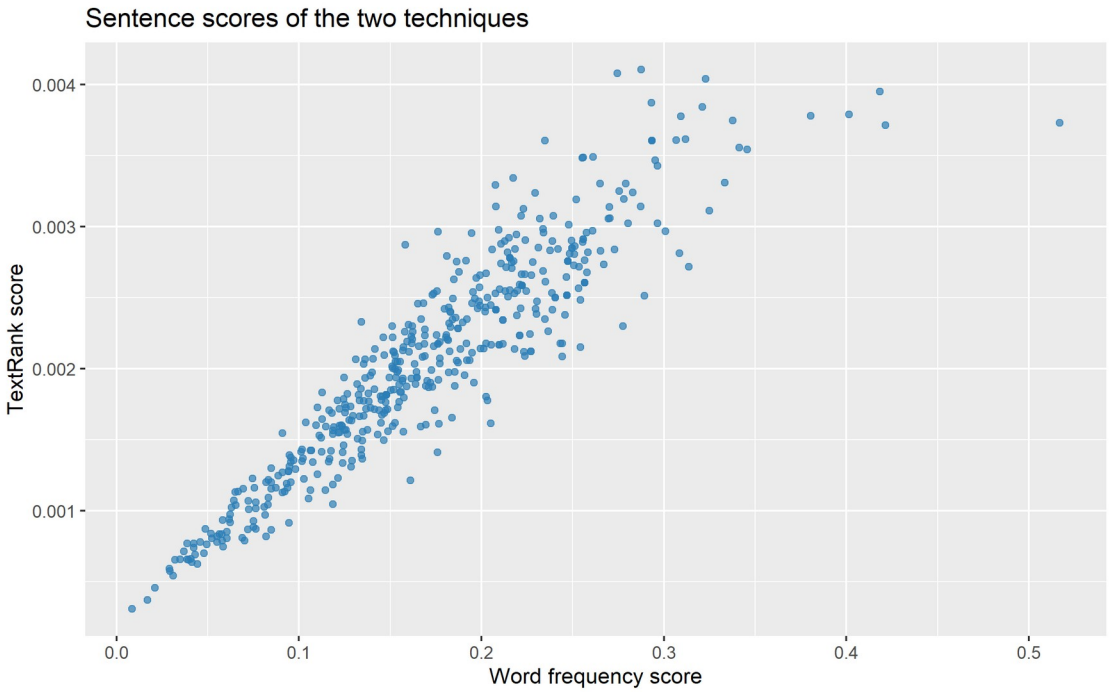


Figure 16: Sentence scores of the two techniques compared

The scatter plot shows each of the 485 sentences as one point, with the word frequency score on the horizontal axis and the TextRank score on the vertical axis. The points rise from the bottom left to the top right, so a sentence that scores well with one technique usually scores well with the other. The spread at the top right shows where they differ: sentences with the highest word frequency scores (above 0.4, such as the one near 0.5) do not always get the highest TextRank scores, which is why the two summaries are different.

Question 4: ETL with R and PostgreSQL

4.1 Extract: web scraping and regex

The qualifications page shows one card per programme. Each card holds a programme type, a faculty, a title, a shortened description, the learning modes and a start date. The script reads the 65 cards and builds a data frame. Regex is then used to extract the start date from the text "Start date: 15 February 2027", and to create two numerical attributes from the text: the number of learning modes (n_modes) and the number of words in the description (desc_words).

```
library(rvest)
library(httr)
library(stringr)

ua <- user_agent("EduvosStudentProject/1.0 (non-commercial, educational)")
Sys.sleep(2)
page <- read_html(GET("https://www.eduvos.com/qualifications/", ua))
cards <- html_elements(page, "article.qf-card")            # one card per programme

raw <- data.frame(
  title          = html_text2(html_element(cards, ".qf-card-title")),
  faculty        = html_attr(cards, "data-faculty"),
  programme_type = html_attr(cards, "data-programme-type"),
  learning_modes = html_attr(cards, "data-learning-mode"),
  description    = html_text2(html_element(cards, ".qf-card-description")),
  start_text     = html_text2(html_element(cards, ".qf-start-date"))
)

# regex: pull the date out of "Start date: 15 February 2027"
raw$start_date <- str_extract(raw$start_text, "\\d{1,2} [A-Za-z]+ \\d{4}")
# regex: count the learning modes in the text (numerical attribute, inferred)
raw$n_modes <- str_count(raw$learning_modes, "(?i)full[- ]time|part[- ]time|remote learning")
# regex: count the words of the description (numerical attribute, inferred)
raw$desc_words <- str_count(raw$description, "\\S+")

cat("Records scraped:", nrow(raw), "\n")
print(table(raw$learning_modes))                           # shows the inconsistent labels
cat("Descriptions ending in '...':", sum(str_detect(raw$description, "\\.\\.\\.$")), "\n")
cat("Empty learning modes:", sum(raw$learning_modes == ""), "\n")
write.csv(raw, "Q4_1_raw.csv", row.names = FALSE)
```

The data frame has these columns:

Column	Type	Content
title, faculty, programme_type	Text	Name, faculty and type of the programme
learning_modes, description	Text	Free text from the card
start_date	Date	Extracted with regex from the start date text
n_modes	Number (inferred)	Count of the learning modes in the text, by regex
desc_words	Number (inferred)	Count of the words in the description, by regex

Output of the scrape on my computer:

```
Records scraped: 65

Campus Full Time                                          3
Campus Full Time, Online Full Time, Online Part Time     40
Distance (Online) Full Time, Distance (Online) Part-Time   1
Distance Full Time                                         1
Distance Full Time, Distance Part Time                     1
```

```
Online Full Time, Online Part Time                              8
Remote Learning                                                10
(empty)                                                         1

Descriptions ending in '...': 64
Empty learning modes: 1
```

The first rows of the raw data frame (Q4_1_raw.csv):

Title	Type	n_modes	desc_words	start_date
Bachelor of Arts Access Programme: ENG	Access	3	19	15 February 2027
Bachelor of Arts Honours in Graphic Design	Honours	2	24	15 February 2027
Bachelor of Arts in Graphic Design	Degree	3	22	15 February 2027
Bachelor of Science in IT: Security and Network Engineering	Degree	0	17	15 February 2027
Short Course in Managing Teams	Corporate Training	1	20	11 January 2027

The raw data has the problems that real web data has:

- Spurious text: 64 of the 65 descriptions are cut off by the website with "...", and the start date comes as the text "Start date: 15 February 2027".
- Inconsistent labels: the same idea is written in different ways ("Distance", "Distance (Online)" and "Online"; "Part-Time" and "Part Time"), and the long type name "Postgraduate Certificates & Diplomas".
- Incomplete data: one programme (the BSc in IT: Security and Network Engineering) has an empty learning mode, so its n_modes is 0.
- Repeated content: the description of the Higher Certificate in Cyber Security is a copy of the description of Cloud and Network Engineering on the website, and the five Pre-Degree programmes show the same description text.

4.2 Transform: cleaning, denoising and imputation

```
clean <- raw
clean$start_text <- NULL

# denoise: remove the "..." the website adds when it cuts a description short
clean$description <- str_trim(str_remove(clean$description, "\\s*\\.\\.\\.$"))

# make the labels consistent: Distance and Distance (Online) are online, Part-Time is Part
Time
clean$learning_modes <- str_replace_all(clean$learning_modes, "Distance \\(Online\\)|
Distance", "Online")
clean$learning_modes <- str_replace_all(clean$learning_modes, "Part-Time", "Part Time")
clean$programme_type <- str_replace(clean$programme_type, "Postgraduate Certificates &
Diplomas", "Postgraduate")

# missing value: an empty learning mode becomes NA and is imputed with the most
# common learning mode of the same programme type
clean$learning_modes[clean$learning_modes == ""] <- NA
most_common <- function(x) names(which.max(table(x)))
by_type <- tapply(clean$learning_modes, clean$programme_type, most_common)
missing <- is.na(clean$learning_modes)
clean$learning_modes[missing] <- by_type[clean$programme_type[missing]]

# recalculate the numerical columns on the cleaned text and fix the date type
clean$n_modes <- str_count(clean$learning_modes, "(?i)full time|part time|remote learning")
clean$desc_words <- str_count(clean$description, "\\S+")
clean$start_date <- as.Date(clean$start_date, format = "%d %B %Y")
clean <- unique(clean)                                    # remove duplicate records

cat("Records after cleaning:", nrow(clean), "\n")
print(table(clean$learning_modes))
cat("Missing values left:", sum(is.na(clean)), "\n")
write.csv(clean, "Q4_2_cleaned.csv", row.names = FALSE)
```

The cleaning steps:

- Denoising: the "..." is removed from the descriptions, and the start date text is turned into a real date.
- Consistency: "Distance (Online)" and "Distance" become "Online", "Part-Time" becomes "Part Time", and the long type name is shortened to "Postgraduate". The numerical columns are calculated again on the cleaned text.
- Imputation: the empty learning mode is changed to NA and filled with the most common learning mode of the same programme type. The BSc in IT: Security and Network Engineering is a Degree, so it received "Campus Full Time, Online Full Time, Online Part Time" and its n_modes changed from 0 to 3.
- Duplicates: unique() confirmed that there are no duplicate records.

The learning modes before and after cleaning:

Learning modes (cleaned label)	Before	After	Change
Campus Full Time	3	3	none
Campus Full Time, Online Full Time, Online Part Time	40	41	1 imputed
Online Full Time	0	1	from "Distance Full Time"
Online Full Time, Online Part Time	8	10	2 relabelled
Remote Learning	10	10	none
Empty or other labels	4	0	cleaned

After cleaning there are 65 records and no missing values. The cleaned data is exported as Q4_2_cleaned.csv, with 8 columns: title, faculty, programme_type, learning_modes, description, start_date, n_modes and desc_words.

4.3 Load and analyse in PostgreSQL

The table is created in the Project_2 schema, with the title as the primary key, and the cleaned CSV is imported with the pgAdmin Import/Export tool (CSV format, header on).

```
-- Question 4.3: import the cleaned CSV into PostgreSQL and analyse it

-- 1. Create the table in the Project_2 schema
CREATE SCHEMA IF NOT EXISTS "Project_2";
DROP TABLE IF EXISTS "Project_2".programmes;
CREATE TABLE "Project_2".programmes (
    title          TEXT PRIMARY KEY,
    faculty        TEXT,
    programme_type TEXT,
    learning_modes TEXT,
    description    TEXT,
    start_date     DATE,
    n_modes        INTEGER,
    desc_words     INTEGER
);

-- 2. Import Q4_2_cleaned.csv with the pgAdmin Import/Export tool (csv, header on),
--    or with: COPY "Project_2".programmes FROM 'path/to/Q4_2_cleaned.csv' WITH (FORMAT csv,
HEADER true);
```

After the import, a check query counts the records in the table. It returns the 65 programmes of the cleaned data set.

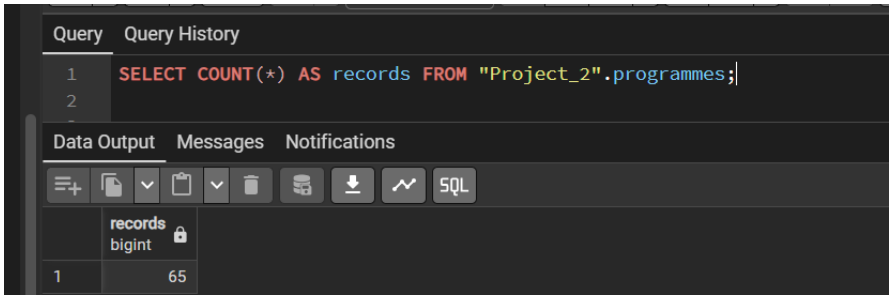


Figure 17: Check query after the import, 65 records

Investigation 1: which faculties offer the most programmes, and how flexible are they?

The query counts the programmes per faculty, averages the numerical attribute n_modes, and uses the window function RANK() to rank the faculties by size.

```sql
-- Query 1: how many programmes does each faculty offer, and how flexible are they?
-- RANK() is a window function that ranks the faculties by number of programmes
SELECT faculty,
       COUNT(*)                    AS programmes,
       ROUND(AVG(n_modes), 2)      AS avg_learning_modes,
       RANK() OVER (ORDER BY COUNT(*) DESC) AS size_rank
FROM "Project_2".programmes
GROUP BY faculty
ORDER BY size_rank;
```

Data Output | Messages | Notifications

Showing rows: 1 to 6

	faculty text	programmes bigint	avg_learning_modes numeric	size_rank bigint
1	Technology	20	2.80	1
2	Commerce	18	2.28	2
3	Humanities and A...	15	2.73	3
4	Business School	5	1.00	4
5	Applied Sciences	4	1.50	5
6	Law	3	2.67	6

Figure 18: Query 1 and its result

Interpretation: Technology (20 programmes) and Commerce (18) are the largest faculties and together offer 38 of the 65 programmes (58%). Technology is also the most flexible faculty, with 2.80 learning modes on average, which means that most of its programmes can be studied on campus, online full time and online part time. Business School has the lowest average (1.00) because all of its 5 programmes are short courses with only remote learning. Applied Sciences is next (1.50), because three of its four programmes (Biomedicine, Biotechnology Management and the Higher Certificate in Bioscience) are offered on campus full time only.

Investigation 2: which programme types can be studied part time?

The query uses the case-insensitive POSIX regex operator ~* on the learning modes text to count the programmes that offer part time study for each programme type.

```sql
-- Query 2: which programme types can be studied part time?
-- ~* is the case-insensitive POSIX regular expression match
SELECT programme_type,
       COUNT(*) AS programmes,
       COUNT(*) FILTER (WHERE learning_modes ~* 'part time') AS part_time,
       ROUND(100.0 * COUNT(*) FILTER (WHERE learning_modes ~* 'part time') / COUNT(*)) AS pct_part_time
FROM "Project_2".programmes
GROUP BY programme_type
ORDER BY pct_part_time DESC, programmes DESC;
```

Data Output | Messages | Notifications

Showing rows: 1 to 8 | Page No: 1

	programme_type text	programmes bigint	part_time bigint	pct_part_time numeric
1	Honours	6	6	100
2	Masters	5	5	100
3	Pre-Degree	5	5	100
4	Access	4	4	100
5	Degree	21	19	90
6	Higher Certificate	10	9	90
7	Postgraduate	4	3	75
8	Corporate Training	10	0	0

Figure 19: Query 2 and its result

Interpretation: all Honours, Masters, Pre-Degree and Access programmes (100%) can be studied part time, which suits working students and students who are preparing to enter a degree. Degrees (19 of 21) and Higher Certificates (9 of 10) are close behind at 90%, and the exceptions are the three campus only science programmes. Postgraduate certificates and diplomas reach 75% because the Postgraduate Diploma in Accounting is offered online full time only. Corporate Training is the only type with no part time option (0 of 10), because the short courses are delivered by remote learning. The query only works because the cleaning in 4.2 made "Part-Time" and "Part Time" the same label.

4.4 GitHub repository

All the work for Questions 1 to 4 is in one GitHub repository: https://github.com/Opzalas369/ITBDA4-14_Big_Data_Analytics_Project_2

The repository has this structure:

```
Project_2/
  README.md                 what the project is and how to run it
  docs/                     the project document (Questions 1 to 4)
  datasets/                 the data supplied with the project
    map_reduce_test_docs/   doc1.txt to doc7.txt (Question 1)
    sa_news/                news_1.txt to news_10.txt (Question 2)
  Q1/                       Java code (Mapper, Reducer, Driver), run steps and outputs
  Q2/                       notebook, data frame, topic tables and charts
  Q3/                       notebook, scraped data, summaries and charts
  Q4/                       notebook, raw and cleaned data, PostgreSQL script
```

The documentation is in the docs folder and the data sets are in the datasets folder and in the question folders (the data that the scrapes in Questions 3 and 4 produced). The notebooks in Q2, Q3 and Q4 are R notebooks that show the flow of the analysis with their output. Question 1 is Java code, as the question asks, so it has the code files, the run steps and the real output of the job instead of a notebook. The repository is public so that it can be marked.